

Notas de Aula – Guilherme Sipahi – Arquitetura de Computadores

Compilação

Programa Alto nível → Compilador → Programa Baixo nível

Exemplo:

```
BEGIN;  
    A=B+C  
    D=A*3  
END;
```

Fases da Compilação:

- 1) Análise Léxica
 - classifica as palavras em várias categorias
 - a. palavras-chave: if, then, while, do
 - b. identificadores: x, volume_total
 - c. operadores: =, <, >, +, *
 - d. delimitadores: vírgulas e parênteses
- 2) Análise Sintática
 - junta as palavras em estruturas sintáticas
 - B+C é uma expressão
 - várias expressões formam um comando ('statement')
- 3) Gerador de código intermediário
 - gera uma sequência de instruções simples num formato pré-definido (numa linguagem próxima ao assembler).
- 4) Otimizador
 - tenta melhorar o código produzido no 3º estágio.
- 5) Geração de código
 - gera o código em assembler

Exemplos (Dowd & Severance):

O código em fortran:

A = -B + C * D / E

É reescrito, quebrando em expressões mais básicas, como:

T1 = D / E

T2 = C * T1

T3 = -B

A = T3 + T2

Compilação (Um pouco mais de detalhes)

1 - Análise Léxica

- lê o conjunto de símbolos até encontrar um delimitador ou formar uma palavra que seja uma diretiva da linguagem.
- delimitador pode ser um separador (vírgula ou espaço) ou um operador.

2 - Criação da Tabela de símbolos:

Endereço na Tabela	Nome do Campo	Tamanho	Modo
001	A	1	Binário

3 - Análise Sintática das declarações

4 - Processamento do lado esquerdo (atribuição)

Pilha de Operandos	Pilha de Operador	Atribuição
001	=	A=

5 - Processamento do lado direito

Endereço	Nome	Tamanho	Modo	Endereçamento relativo
001	A	1	Binário	não-designado
002	B	1	Binário	não-designado
003	C	1	Binário	não-designado

Pilha de Operandos	Pilha de Operadores	
003		C
002	+	B+
001	=	A=

6 - Forma Intermediária (Quádruplas)

Pilha de Operandos	Pilha de Operadores	Quádrupla
R1		R1: ADD 003 002
001	=	

R1: ADD 003 002
STO R1 001

Na próxima declaração:

R1: ADD 003 002
STO R1 001
R2: MPY 001 "3"

STO R2 005

Tabela:

Endereço	Nome	Tamanho	Modo	End. Rel
001	A	1	binário	not
002	B	1	binário	not
003	C	1	binário	not
004	"3"	1	binário	not
005	D	1	binário	not

7 - Geração do código

Posição	Instrução	Quádrupla	
000	LOAD 003	R1: ADD 003 002	
001	ADD 002		
002	STO 001	STO R1 001	A = B +C
003	LOAD 001		
004	MPY "3"	R2 MPY 001 "3"	
005	STO 005	STO R2 005	D = A * 3

8 - Otimização

- Se A não for utilizado novamente (neste código ele não é)
Então:

Posição	
0000	LOAD 003
0001	ADD 002
0002	MPY "3"
0003	STO 005

Assembler não otimizado:

0000	LOAD 0007	
0001	ADD 0008	
0002	STO 0009	
0003	LOAD 0009	
0004	MPY "3"	
0005	STO 0010	
	BRANCH	
0006	XXXX	(desvio incondicional)
0007	(valores de B)	
0008	(valores de C)	
0009	(valores de A)	
0010	(valores de D)	